



Programming Java

Creating Classes

Incheon Paik



Contents

- The General Form of a Class
- Creating Simple Classes
- Adding Constructors
- Constructor Overloading
- The this Keyword
- Instance Variables and Instance Methods
- Static Variables and Static Methods
- Local Variables and Variable Scope
- Method Overloading
- Argument Passing
- Generic Class Types

The General Form of a Class

Declaration of a Class

```
Class clsName {  
    // instance variables  
    Type1 varName1 = value1;  
    Type2 varName2 = value2;  
    .....  
    typeN varNameN = valueN;  
  
    //Constructors  
    clsName(cparams1) {  
        // body of constructor  
    }  
    .....  
    clsName(cparamsN) {  
        // body of constructor  
    }  
  
    // Methods  
    Rtype1 mthName1(mparams1) {  
        // body of method  
    }  
    .....  
    Rtype1 mthName1(mparams1) {  
        // body of method  
    }  
}
```

Creating Simple Classes

A Simple Form

```
class Sample {  
    int a;  
    int b;  
    int c;  
}
```

```
Sample one = new Sample();
```

```
class Point3D {  
    double x;  
    double y;  
    double z;  
}
```

```
Class Point3DExample {  
    public static void main(String args[]) {  
        Point3D p = new Point3D();  
        p.x = 1.1;  
        p.y = 3.4;  
        p.z = -2.8;
```

```
        System.out.println("p.x = " + p.x);  
        System.out.println("p.y = " + p.y);  
        System.out.println("p.z = " + p.z);  
    }  
}
```

```
Result :  
p.x = 1.1  
p.y = 3.4  
p.z = -2.8
```



Adding Constructors

```
class Point3D {  
    double x;  
    double y;  
    double z;
```

Constructor

```
    Point3D (double ax, double zy, double az) {  
        x = ax;  
        y = ay;  
        z = az;  
    }  
}
```

```
Class Point3DExample {  
    public static void main(String args[]) {  
        Point3D p = new Point3D(1.1, 3.4, -2.8);  
        System.out.println("p.x = " + p.x);  
        System.out.println("p.y = " + p.y);  
        System.out.println("p.z = " + p.z);  
    }  
}
```

Result :

```
p.x = 1.1  
p.y = 3.4  
p.z = -2.8
```



Constructor Overloading

Signature : The information to distinguish methods such as the method name, no. of parameters, data types of parameters, the return type

```
class Point3D {  
    double x;  
    double y;  
    double z;  
    Point3D (double ax) {  
        x = ax;  
        y = 1;  
        z = 1;  
    }  
    Point3D (double ax, double zy) {  
        x = ax;  
        y = ay;  
        z = 1;  
    }  
    Point3D (double ax, double zy, double az) {  
        x = ax;  
        y = ay;  
        z = az;  
    }  
}
```

```
class Point3DExample {  
    public static void main(String args[]) {  
        Point3D p = new Point3D(1.1);  
        System.out.println("p.x = " + p.x);  
        System.out.println("p.y = " + p.y);  
        System.out.println("p.z = " + p.z);  
        Point3D p = new Point3D(1.1, 3.4);  
        System.out.println("p.x = " + p.x);  
        System.out.println("p.y = " + p.y);  
        System.out.println("p.z = " + p.z);  
        Point3D p = new Point3D(1.1, 3.4, -2.8);  
        System.out.println("p.x = " + p.x);  
        System.out.println("p.y = " + p.y);  
        System.out.println("p.z = " + p.z);  
    }  
}
```

The this Keyword

this Keyword

this.varname

Invocation of Constructors

this(args);

```
class Point3D {  
    double x;  
    double y;  
    double z;  
  
    Point3D(double x, double y, double z) {  
        this.x = x;  
        this.y = y;  
        this.z = z;  
    }  
}
```

```
class ThisKeywordDemo {  
    public static void main(String args[]) {  
        Point3D p = new Point3D(1.1, 3.4, -2.8);  
        System.out.println("p.x = " + p.x);  
        System.out.println("p.y = " + p.y);  
        System.out.println("p.z = " + p.z);  
    }  
}
```

Instance Variables & Instance Methods

Declaration of an Instance Variable

Type varName1;

Declaration of Multiple Instance Variables

Type varName1, varname2, ... varNameN;

Declaration of an Instance Variable & Initialization

Type varName1 = expr1;

Declaration of Multiple Instance Variables & Initialization

Type varName1, varname2 = expr2, ... varNameN;

Declaration of an Instance Method

```
rtype mthName (mparams) {  
    // body of method  
}
```

Refer to the
supplement
material for more
detailes



Instance Variables & Instance Methods

```
class Bag {
    boolean flag;
    int i, j=2, k=3, l, m;
    double array[] = {-3.4, 8.8E100, -9.2E-100 };
    String s1, s2= new String("Hello");
}

class BagTest {
    public static void main(String args[]) {
        Bag bag = new Bag();
        System.out.println(bag.flag);
        System.out.println(bag.i);
        System.out.println(bag.j);
        System.out.println(bag.k);
        System.out.println(bag.l);
        System.out.println(bag.m);
        for (int i=0; i < bag.array.length; i++)
            System.out.println(bag.array[i]);
        system.out.println(bag.s1);
        system.out.println(bag.s2);
    }
}
```

Result :

```
false
0
2
3
0
0
-3.4
8.8E100
-9.2E-100
null
Hello
```



Instance Variables & Instance Methods

```
class Point3D {
    double x;
    double y;
    double z;

    Point3D (double ax) {
        x = ax;
        y = 1;
        z = 1;
    }
    Point3D (double ax, double ay) {
        x = ax;
        y = ay;
        z = 1;
    }
    Point3D (double ax, double ay, double az) {
        x = ax;
        y = ay;
        z = az;
    }
    // Instance Method
    void move(double x, double y, double z) {
        this.x = x;
        this.y = y;
        this.z = z;
    }
}
```

```
Class Point3DMethod {
    public static void main(String args[]) {
        Point3D p = new Point3D(1.1, 3.4, -2.8);
        System.out.println("p.x = " + p.x);
        System.out.println("p.y = " + p.y);
        System.out.println("p.z = " + p.z);
        p.move(5,5,5);
        System.out.println("p.x = " + p.x);
        System.out.println("p.y = " + p.y);
        System.out.println("p.z = " + p.z);
    }
}
```

Result :

```
p.x = 1.1
p.y = 3.4
p.z = -2.8
p.x = 5.0;
p.y = 5.0;
p.z = 5.0;
```

Static Variables & Static Methods

Declaration of a Static Variable

```
static type varName1;
```

Declaration of Multiple Static Variables

```
static type varName1, varName2, ... varNameN;
```

Refer to the
supplement
material for more
details

Declaration of a Static Variable & Initialization

```
static type varName1 = expr1;
```

Declaration of Multiple Instance Variables & Initialization

```
static type varName1, varName2 = expr2, ... varNameN;
```

The Static Initialization Block

```
class clsName {  
    static {  
        // block of statement  
    }  
}
```

Static Variables & Static Methods

Declaration of a Static Method

```
static rtype mthName(mparams) {  
    // body of method  
}
```

```
class StaticBag {  
    static boolean flag;  
    static int i, j=2, k=3, l, m;  
    static double array[] = {-3.4, 8.8e100, -9.2e-100};  
    static String s1, s2= new String("Hello");  
}  
class BagTest {  
    public static void main(String args[]) {  
        Bag bag = new Bag();  
        System.out.println(StaticBag.flag);  
        System.out.println(StaticBag.i);  
        System.out.println(StaticBag.m);  
        for (int i=0; i < bag.array.length; i++)  
            System.out.println(StaticBag.array[i]);  
        System.out.println(StaticBag.s1);  
        System.out.println(StaticBag.s2);  
    }  
}
```

```
class Thing {  
    static int count;  
    String name;  
  
    Thing(String name) {  
        this.name = name;  
        ++count;  
    }  
}  
//  
class StaticVariable {  
    public static void main(String args[]) {  
        Thing t1 = new Thing("Bowling Ball");  
        System.out.println(t1.name + " " + t1.count);  
        Thing t2 = new Thing("Ping Pong Ball");  
        System.out.println(t2.name + " " + t2.count);  
        Thing t3 = new Thing("Football");  
        System.out.println(t3.name + " " + t3.count);  
    }  
}
```

Result :

```
Bowling Ball 1  
Ping Pong Ball 2  
Football 3
```

Static Variables & Static Methods

```
class X {
    static int array[];
    static {
        array = new int[6];
        for (int i = 0; i < 6; i++)
            array[i] = i;
    }
    class StaticInitializationBlock {
        public static void main(String args[]) {
            for (int i=0; i < 6; i++)
                System.out.println(X.array[i]);
        }
    }
}
```

Result :

0
1
2
3
4
5

```
class LinearEquation {
    static double solve(double a, double b) {
        return -b / a;
    }
}
class StaticInitializationBlock {
    public static void main(String args[]) {
        System.out.println(LinearEquation.solve(2,2));
    }
}
```

Result :

-1.0

Refer to the
supplement
material for more
details

Local Variables and Variable Scope

```
class X {
    void f() {
        for( int j=0; j<5; j++) {
            int k=100;
            System.out.println("j= " + j + "; k= " + k);
        }
    }
}
class VariableScope {
    public static void main(String args[]) {
        X x = new X();
        x.f();
    }
}
```

Result :

j = 0; k = 100
j = 1; k = 100
j = 2; k = 100
j = 3; k = 100
j = 4; k = 100

```
class MyObject {
    static short s = 400; // static variable
    int i = 200;
    void f() {
        System.out.println("s = " + s);
        System.out.println("i = " + i);
        short s = 300; // local variable
        short i = 100; // local variable
        double d = 1E100; // local variable
        System.out.println("s = " + s);
        System.out.println("i = " + i);
        System.out.println("d = " + d);
    }
}
class LocalVariables {
    public static void main(String args[]) {
        MyObject myObject = new MyObject();
        myObject.f();
    }
}
```

Result :

s = 400
i = 200
s = 300
i = 100
d = 1.0E100

Method Overloading

```
class Point3D {
    double x;
    double y;
    double z;
    Point3D (double x) {
        this(x,0,0);
    }
    Point3D (double x, double y) {
        this(x,y,0);
    }
    Point3D (double x, double y, double z) {
        this.x = x;
        this.y = y;
        this.z = z;
    }
    void move(double x, double y, double z) {
        this.x = x;
        this.y = y;
        this.z = z;
    }
    void move(double x, double y) {
        this.x = x;
        this.y = y;
    }
    void move(double x) {
        this.x = x;
    }
}
```

Overloaded Methods

```
Class Point3DOverloadMethods {
    public static void main(String args[]) {
        Point3D p = new Point3D(1.1, 3.4, -2.8);
        p.move(5);
        System.out.println("p.x = " + p.x);
        System.out.println("p.y = " + p.y);
        System.out.println("p.z = " + p.z);
        p.move(6,6);
        System.out.println("p.x = " + p.x);
        System.out.println("p.y = " + p.y);
        System.out.println("p.z = " + p.z);
        p.move(7,7,7);
        System.out.println("p.x = " + p.x);
        System.out.println("p.y = " + p.y);
        System.out.println("p.z = " + p.z);
    }
}
```

Result :

p.x = 5.0	p.x = 7.0
p.y = 3.4	p.x = 7.0
p.z = -2.8	p.x = 7.0
p.x = 6.0;	
p.y = 6.0;	
p.z = -2.9;	

Argument Passing

Call by Value

```
class ArrayArgument {
    public static void main(String args[]) {
        // variable initialization
        int x[] = {11, 12, 13, 14, 15};
        //variable display
        display(x);
        // method invocation
        change(x);
        // variable display
    } // end of main
    public static void change(int x[]) {
        int y[] = {21,22,23,24,25};
        x = y;
    }
    public static void display(int x[]) {
        for (int i=0; i < x.length; i++)
            System.out.print(x[i] + " ");
    } // end of class
} // end of class
```

Result :

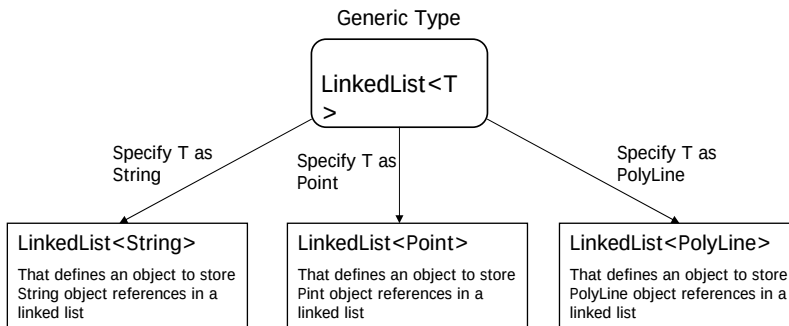
```
11 12 13 14 15
11 12 13 14 15
```

Refer to the
supplement
material for more
details



Generic Class Types

Generic Type : Referred to as **Parameterized Type** , a class or interface type definition that has one or more type parameters. Define an actual class or interface type from a generic type by supplying a type argument for each of the type parameters that the generic type has.



Defining a Generic Class Type

Defining a Generic LinkedList Class

```
public class LinkedList<T> {
    // Generic Type Definition...
}
```

A LinkedList Case

Instantiating a Generic Type

```
LinkedList<String> strings;
LinkedList<String> strings = new LinkedList<String>();
LinkedList<LinkedList<String>> texts = LinkedList<LinkedList<String>> ();
```

Generic Types and Generic Interfaces

```
public class MyClass<T> implements MyInterface<T> {
    // Details of the generic type definitions
}
```

```
import java.util.Iterator;

public class LinkedList<T> implements Iterable<T> {
    public Iterator<T> iterator() { // Code to return a reference to an iterator for this list... }
    // Rest of the Linked List<T> generic type definition as before ...
}
```

Defining a Generic Class Type

Code Examples of LinkedList and PolyLine

<http://ebiz.u-aizu.ac.jp/~paikic/lecture/2005-1/code-examples/Java2-1.5/Code/Ch13/TryGenericLinkedList/LinkedList.java>

<http://ebiz.u-aizu.ac.jp/~paikic/lecture/2005-1/code-examples/Java2-1.5/Code/Ch13/TryGenericLinkedList/Point.java>

<http://ebiz.u-aizu.ac.jp/~paikic/lecture/2005-1/code-examples/Java2-1.5/Code/Ch13/TryGenericLinkedList/PolyLine.java>

<http://ebiz.u-aizu.ac.jp/~paikic/lecture/2005-1/code-examples/Java2-1.5/Code/Ch13/TryGenericLinkedList/TryGenericLinkedList.java>

Code Examples of LinkedList, Collection-Based for Loop, and AutoBoxing

<http://ebiz.u-aizu.ac.jp/~paikic/lecture/2005-1/code-examples/Java2-1.5/Code/Ch13/EnablingForLoop/LinkedList.java>

<http://ebiz.u-aizu.ac.jp/~paikic/lecture/2005-1/code-examples/Java2-1.5/Code/Ch13/EnablingForLoop/Point.java>

<http://ebiz.u-aizu.ac.jp/~paikic/lecture/2005-1/code-examples/Java2-1.5/Code/Ch13/EnablingForLoop/PolyLine.java>

<http://ebiz.u-aizu.ac.jp/~paikic/lecture/2005-1/code-examples/Java2-1.5/Code/Ch13/EnablingForLoop/TryAutoboxing.java>

Exercise

■ Step 1 (Make a Class Method)

- You can fill the method as the fashion of other methods.

■ Step 2 (Call by Reference)

Write a class for “call by referencing”.

```
public static void swap(Swap obj) {  
    // Object variable swapping  
}
```

■ Step 3 Static Variables & the finalize Method

- Need to understand static variables : Static Variables are shared among objects
- When some objects are not used any more and the garbage collector is invoked, the finalized method of the object will be invoked before the garbage collector reclaim the memory.



Exercise

Step 4, A Static Inner Class

- Able to use static variables

OuterClass.InnerClass

```
class OuterClass {
    // ...
    static class InnerClass {
        static int staticVariable;
        // ...
    }
}
```

- Can refer without creating object

📄 [StaticInnerClass.java]



Exercise

```
class OuterClass {
    static class InnerClass {
        static String str;
        InnerClass(String s) {
            str = s;
        }
        void print() {                // Instance Method
            staticPrint(str);
        }
        static void staticPrint(String s) {    // Static Method
            str = s;
            System.out.println(s);
        }
    } // end of InnerClass
} // end of OuterClass
public class StaticInnerClass {
    public static void main(String[] args) {
        String s = "... without creating Outer-class object";
        OuterClass.InnerClass p = new OuterClass.InnerClass(s);
        p.print();
        OuterClass.InnerClass.staticPrint("call static method");
        p.print();
    }
}
```

Guide:

When you solve this problem, you can use this example directly. Some error messages from compiler may give you confusion. Therefore, refer to this example carefully.

Run:

C:\> java StaticInnerClass



Exercise

■ Step 5 (Generic Class Types)

- Related Slides : #17,18,19